QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team InvAgent
Team Repo URL	https://github.com/cendoru/LailatulCoders
Project Board URL	https://trello.com/invite/b/69ed01a046b5a4e8da68fc54/ATTIc842c2059bf6ca78b740e004b2339789FF2734E2/invagent-by-lailatulcoders
Live Deployment URL (optional for preliminary round)	*Not Applicable.*

Objective:

The objective of InvAgent is to develop an AI-driven inventory and procurement system that automates decision-making processes using a structured agent-based architecture.

Instead of relying on manual operations, the system enables a natural language interface where user requests are interpreted by a reasoning engine (Gemini LLM), which determines and executes appropriate actions such as product lookup, supplier selection, stock validation, and restocking.

The system aims to:

- *Streamline inventory management through automated stock operations*
- *Improve procurement decisions by selecting optimal suppliers based on defined criteria (price, rating, lead time)*
- *Ensure consistency and reliability through a controlled agent loop with structured JSON outputs*
- *Maintain system robustness via validation, error handling, and fallback mechanisms*
- *Integrate AI reasoning with deterministic backend services in a secure and testable architecture*

Ultimately, InvAgent bridges the gap between unstructured user input and structured enterprise operations by combining LLM reasoning with traditional backend service orchestration.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns testing with existing code components via the Requirement Traceability Matrix, ensuring every API endpoint and service method is tested against requirements.

1.1 In-Scope Core Features

© UMHackathon 2026

Email: umhackathon@um.edu.my

Inventory Management

- Add stock to existing products based on incoming supply
- Remove stock when inventory is consumed or dispatched
- Validate stock levels to prevent invalid or negative inventory states
- Determine whether a product requires restocking based on predefined thresholds

Supplier Management

- Retrieve all suppliers associated with a specific product
- Select the best supplier based on predefined criteria such as rating, lead time, and pricing
- Maintain product-supplier relationships through a structured mapping system

Procurement Workflow

- Trigger manual restocking for a specific product
- Automatically initiate restocking through a scheduled background process
- Generate procurement actions based on inventory state and supplier availability

AI Agent Orchestration Layer

- Interpret natural language user input into structured system actions
- Use Gemini LLM to determine the next step in the workflow
- Support multi-step reasoning via an iterative agent loop
- Produce structured JSON outputs for deterministic backend execution
- Maintain state across steps such as product selection, supplier resolution, and tool execution results

System State Management

- Track user requests through a centralized state object across the agent loop
- Store intermediate outputs including selected product, supplier, and tool responses
- Ensure continuity between LLM reasoning and backend service execution

1.2 Out-of-Scope

- Predictive demand forecasting or demand prediction using historical sales data
- Multi-warehouse or multi-location inventory synchronization and conflict resolution
- Integration with IoT devices, sensors, or real-time physical inventory tracking systems
- Direct payment processing, billing, or financial transaction handling
- Full enterprise ERP integration (limited to simulated or modular procurement workflows only)
- Advanced supply chain optimization beyond supplier selection logic (e.g., logistics routing, fleet management)

© UMHackathon 2026

Email: umhackathon@um.edu.my

- External marketplace integration for automated purchasing from third-party platforms.

2. Risk Assessment & Mitigation Strategy

Quality Assurance risks are anticipatory. Below are the technical risks associated with InvAgent's requirements and architecture (GLM reasoning engine + multi-API orchestration + stateful workflows). Each risk is evaluated using the 5×5 Risk Assessment Matrix, where Risk Score = Likelihood × Severity.

Technical Risk	**Likelihood (1–5)**	**Severity (1–5)**	**Risk Score (L×S)**	**Mitigation Strategy**	**Testing Approach**
Hardcoded logic in restock workflow	*5*	*4*	*20 (Critical)*	*Replace hardcoded product selection with dynamic queries and configurable logic*	*Test with multiple products; verify only product ID 1 gets restocked*
Missing or weak input validation in controllers	*4*	*4*	*16 (Critical)*	*Add request validation and enforce service-layer checks*	*Send invalid inputs (null, negative values) and verify rejection*
Null values in different fields	*4*	*3*	*12 (High)*	*Apply null handling and default fallback values*	*Test products with deliberate missing fields and verify safe execution*
Service-layer unchecked exceptions	*3*	*4*	*12 (High)*	*Centralize exception handling using controller advice or try-catch blocks*	*Simulate service failure and verify error response*
InventoryService methods throw unchecked exceptions	*3*	*4*	*12 (High)*	*Add proper exception handling in controllers*	*Simulate service failures; verify catch blocks execute*
Scheduled task overlap or concurrency issues	*2*	*3*	*6 (Medium)*	*Use fixedDelay scheduling or locking mechanisms*	*Stress test auto-restock execution under repeated triggers*
Missing or incomplete persistence layer consistency	*4*	*5*	*20 (Critical)*	*Ensure proper JPA repositories and transactional integrity*	*Validate data persistence across service and repository layers*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating, and thorough testing is required*
16 – 25	*Critical*	*Priority is Highest. Need an extensive level of testing.*

3. Test Environment & Execution Strategy

This section defines where InvAgent is tested, how test data is managed, and the rules governing each testing phase. Given that InvAgent relies on Z. AI's GLM as the central reasoning engine, the testing strategy accounts for non-deterministic AI outputs, stateful workflows, and API orchestration.

- **Unit Test**
    - *Scope: Controller logic, entity validation, service method logic, state machine transitions*
    - *Framework: JUnit 5 + Mockito for Spring Boot + Spring Boot Test Framework*
    - *Execution: Local development machine, CI pipeline (GitHub Actions), No database or external API dependencies*
    - *Isolation: InventoryService, SupplierService, and ProcurementService are fully mocked, No real Gemini API calls during unit tests, No scheduled tasks executed*
    - *Pass Condition: Achieved when:*
        - ***Happy cases:***
            - *addStock() receives a valid Product object with positive quantity → returns exact string "Stock added successfully"*
            - *removeStock() receives a valid Product object with quantity ≤ current stock → returns exact string "Stock removed successfully"*

© UMHackathon 2026

Email: umhackathon@um.edu.my

    - *getAllSuppliers() receives an existing productId → returns non-null List<Supplier> containing at least one supplier*
    - *getBestSupplier() receives an existing productId → returns non-null Supplier object with valid id, name, rating, leadTime*
    - *restock() receives existing productId where needsRestock() is true and suppliers exist → returns string containing "Restock triggered", supplier name, and quantity*
    - *autoRestock() scheduled task runs → logs "[AUTO RESTOCK] Product: 1 | Supplier: X | Qty: Y" with no exceptions*
- ***Negative cases:***
    - *addStock() receives null Product → returns string starting with "Failed to add stock:."*
    - *addStock() receives negative quantity → returns error message (does NOT call service)*
    - *removeStock() receives quantity exceeding current stock → returns error message (does NOT call service)*
    - *needsRestock() called with GET + @RequestBody → HTTP status 4xx error (method not allowed)*
    - *restock() receives productId with no suppliers → returns exact string "No suppliers available for product ID X"*
    - *getAllSuppliers() receives invalid productId → controller catches exception, logs error, re-throws (does not return null)*
    - *autoRestock() runs with empty supplier list → logs "[AUTO RESTOCK NO SUPPLIER]" warning, no restock executed*
- ***Edge cases:***
    - *addStock() receives quantity = 0 → returns error message (no stock change performed)*
    - *removeStock() receives quantity = 0 → returns error message (no stock change performed)*
    - *needsRestock() called when current stock equals restock threshold → returns false*
    - *needsRestock() called when current stock is less than restock threshold → returns true*
    - *restock() called when needsRestock() returns false → returns "No restock needed for product ID X"*
    - *Product has null stockLevel → treated as 0 for threshold comparison*
    - *Product has null restockThreshold → treated as 0 (always needs restock if stock > 0)*
    - *autoRestock() runs when product has null stockLevel → handled without NullPointerException*
    - *restock() generates random quantity between 10-59 inclusive → quantity is always ≥10 and ≤59*
- ***Return format:***

© UMHackathon 2026

        - *InventoryController methods (addStock, removeStock, restock) return String type with a human-readable message*
        - *SupplierController getAllSuppliers() returns List<Supplier> (JSON array)*
        - *SupplierController getBestSupplier() returns a single Supplier object (JSON object)*
        - *needsRestock() returns a boolean primitive (true/false)*
        - *All error responses from controllers include a descriptive error message from the caught exception*
        - *No controller returns null to the client (either a valid object or an exception thrown)*
    - *No external calls:*
        - *All service dependencies (InventoryService, SupplierService, ProcurementService) are mocked using Mockito @MockBean*
        - *Unit tests execute without a database connection (no @DataJpaTest)*
        - *Unit tests execute without actual GLM API calls*
        - *Unit tests execute without real network requests*
        - *Mocked service methods return predefined values (e.g., when(inventoryService.needsRestock(any())).thenReturn(true))*
        - *No real @Scheduled task execution in unit tests (scheduler disabled)*
        - *Tests complete in < 1 second per test case (no network latency)*

- *Example Unit Tests Written:*

```java
@Test
void testAddStock_Success() {
    Product product = new Product();
    product.setId(1);
    product.setStockLevel(10);

    when(inventoryService.addStock(any(), eq(5))).thenReturn(true);

    String response = inventoryController.addStock(product, 5);
    assertEquals("Stock added successfully", response);
}

@Test
void testAddStock_NullProduct() {
    String response = inventoryController.addStock(null, 5);
    assertTrue(response.contains("Failed to add stock"));
}
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
@Test
void testNeedsRestock_GetWithRequestBody_ShouldReturn400() {
    // This test validates the HTTP method issue
    mockMvc.perform(get("/inventory/restock-check")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"id\":1}"))
            .andExpect(status().is4xxClientError());
}

@Test
void testAutoRestock_OnlyProduct1() {
    // Verify hardcoded behavior
    procurementController.autoRestock();
     verify(procurementService, times(1)).processRestock(argThat(p -> p.getId() ==
1));
}
```

- **Integration Test**
    - *Scope: Controller → Service → Repository (real database)*
    - *Execution: After unit tests pass; runs in staging environment*
    - *Database: H2 in-memory or Testcontainers for MySQL/PostgreSQL*
    - *Pass Condition: End-to-end: HTTP request → controller → service → response matches expected*

Example Integration Tests

```
Java
@Test
void testAddStock_Integration() {
    Product product = productRepository.save(createTestProduct());

    mockMvc.perform(post("/inventory/add")
            .param("quantity", "10")
            .contentType(MediaType.APPLICATION_JSON)
            .content(asJsonString(product)))
            .andExpect(status().isOk())
            .andExpect(content().string("Stock added successfully"));
}

@Test
void testGetBestSupplier_Integration() {
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
    int productId = 1;

    mockMvc.perform(get("/suppliers/{productId}/best", productId))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").exists())
            .andExpect(jsonPath("$.name").isString());
}
```

- **Test Environment (CI/CD Practice):**
  - *Local Environment*
    - *Purpose: Developer testing during development*
    - *Database: H2 in-memory database (no persistence)*
    - *External Dependencies: None - all services are mocked*
    - *Execution: Manual testing on developer's machine*
    - *Speed: Fast (no network calls, no disk I/O)*
  - *Staging / CI Environment*
    - *Purpose: Automated testing in CI pipeline*
    - *Database: Testcontainers with PostgreSQL (real database, ephemeral)*
    - *External Dependencies: Mock GLM API (simulated responses)*
    - *Execution: GitHub Actions or CI runner automatically*
    - *Speed: Moderate (container startup + tests)*
  - *Production Environment*
    - *Purpose: Live system serving real users*
    - *Database: Production database (persistent, backed up)*
    - *External Dependencies: Real supplier APIs (future implementation)*
    - *Execution: Deployed application on production server*
    - *Speed: Production performance with real network calls*

- **Regression Testing & Pass/Fail Rules:**
  - *Execution Phase*
    - Run all test cases on every merge to the main branch
    - Execute regression suite automatically via GitHub Actions CI pipeline
    - Run tests in the staging environment before any production deployment
    - Minimum test count: 15+ test cases covering:
      - Happy path (TC-01 to TC-06: addStock, removeStock, getSuppliers, getBestSupplier, manual restock, auto-restock)
      - Negative/edge cases (TC-07 to TC-15: negative quantity, insufficient stock, HTTP method violations, null products, missing parameters, no suppliers)

© UMHackathon 2026

Email: umhackathon@um.edu.my

        - NFR/performance tests (TC-NFR-01 to TC-NFR-06: concurrent updates, response time, scheduled task overlap, connection pool, logging, memory leaks)
    - *Pass/Fail Condition*
        - API Response matching:
            - addStock() success → must exactly return "Stock added successfully"
            - addStock() failure → must contain "Failed to add stock: "
            - removeStock() success → must exactly return "Stock removed successfully"
            - restock() success → must contain "Restock triggered for product ID X"
            - getAllSuppliers() → must return List<Supplier> JSON array
            - getBestSupplier() → must return single Supplier JSON object
        - HTTP Status Codes:
            - Valid requests → 200 OK
            - Missing @RequestParam → 400 Bad Request
            - GET with @RequestBody on needsRestock() → 400 or 405 Method Not Allowed
            - Exception in controller → error logged but returns 200 with error message (current behavior)
        - State validation (for future GLM integration):
            - Stock level before add: 10 → after add: 10 + quantity
            - Stock level before remove: 10 → after remove: 10 - quantity
            - Remove with quantity > stock → no state change, error returned
            - needsRestock() returns true when stockLevel < restockThreshold
            - autoRestock() only changes state for product ID 1 (hardcoded)
        - No partial passes allowed – any deviation = failed test
        - For future GLM outputs: schema validation + cross-validation against actual API responses required
    - *Continuation Rule*
        - Critical tests must pass before integration tests proceed:
            - TC-01: Add stock to existing product (POST /inventory/add)
            - TC-02: Remove stock from product (POST /inventory/remove)
            - TC-05: Manual restock trigger (POST /procurement/restock/{id})
            - TC-07: Add stock with negative quantity (must fail)
            - LOG-01: Request logging in InventoryController
        - If any critical test fails → pipeline blocks all subsequent test phases
        - No merging to main allowed until critical tests achieve 100% pass rate
        - Integration tests execute ONLY after critical test suite returns green
    - *Additional Rules*
        - Regression failures automatically create defect log entry in GitHub Issues / Jira
        - Failed regression tests trigger automatic rollback of the merge attempt
        - Team lead must manually approve a re-run after fixing regression failures
        - Regression suite timeout: 10 minutes total execution time

© UMHackathon 2026

Email: umhackathon@um.edu.my

        - If suite exceeds 10 minutes → automatically fails
        - Individual test timeout: 30 seconds per test
        - Scheduled test (autoRestock()) cannot be sped up → mock Clock for testing

- **Test Data Strategy:**
    - *Manual Test Data*
        - Mock Inventory Items (5 products):
            - Rice – stock: 10kg, threshold: 30kg
            - Flour – stock: 5kg, threshold: 20kg
            - Oil – stock: 50L, threshold: 30L
            - Sugar – stock: 25kg, threshold: 15kg
            - Salt – stock: 100kg, threshold: 50kg
        - Mock Suppliers (3 vendors):
            - LocalMart – rating: 4.5, lead time: 3 days
            - BulkCo – rating: 4.2, lead time: 5 days
            - FastDistro – rating: 4.8, lead time: 1 day
        - Product-Supplier Pricing:
            - Rice: LocalMart (RM45), BulkCo (RM42), FastDistro (RM48)
            - Flour: LocalMart (RM30), BulkCo (RM28), FastDistro (RM32)
            - Each supplier has a capacity limit: LocalMart (500kg max per order)
        - Test User Roles:
            - Warehouse supervisor – can add/remove stock, check inventory
            - Procurement manager – can approve high-value orders (>RM10,000)
            - Approver – receives escalation notifications
    - *Automated Test Data*
        - SQL Seed Scripts
            - Pre-populates 5 products with initial stock levels
            - Pre-populates 3 suppliers with ratings and lead times
            - Creates ProductSupplier junction table with pricing
            - Sets restock thresholds per product
        - Dynamic Stock Level Injection:
            - Rice: inject 10kg before test run
            - Flour: inject 5kg before test run
            - Oil: inject 50L before test run
            - Sugar: inject 25kg before test run
            - Salt: inject 100kg before test run
        - Threshold Configurations Per Item:
            - Rice: restock when below 30kg
            - Flour: restock when below 20kg
            - Oil: restock when below 30L
            - Sugar: restock when below 15kg
            - Salt: restock when below 50kg
        - Supplier Capacity Limits:
            - LocalMart: max 500kg per order

© UMHackathon 2026

Email: umhackathon@um.edu.my

        - BulkCo: max 1000kg per order
        - FastDistro: max 200kg per order
    - Reset Scripts:
        - Truncates all tables before each test run
        - Re-seeds fresh data from SQL scripts
        - Resets auto-increment sequences
        - Restores stock levels to the original predefined values
        - Runs automatically before every test suite execution

- **Passing Rate Threshold:**
    - *Overall Pass Rate: Minimum 85% of all executed test cases must pass*
    - *Critical Tests (P0): 100% pass rate required – no exceptions*
        - Includes: TC-01, TC-02, TC-05, AI-01, API security, and state loss prevention
    - *High Priority Tests (P1): Minimum 90% pass rate required*
        - *Includes*: Negative/edge cases (TC-07 to TC-15), API failure recovery, missing data handling, supplier selection logic, and stock validation
    - *Non-Functional Tests (NFR): Minimum 80% pass rate required*
        - *Includes: Performance/latency, token cost limits, oversized input handling, concurrent request handling, and scheduled task accuracy*
    - *Blocking Rule: If any critical test fails, the pipeline blocks deployment to production immediately*
    - *Retry Limit: Failed critical tests may be retried once after fix; second failure requires manual review*

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

This threshold is crucial because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	Zero Build Error	0 Errors	Passed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	2 Warnings (non-critical)	Passed (warnings allowed)
Test Coverage	Minimum 80%	82%	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	**Requirements**	**Project**	**Pass/Failed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

Regression Test	Minimum 90%	93%	Passed
API Response Time	<500ms for inventory operations	83%	Passed
Critical Bugs	Zero P0/01 Bugs	0	Passed
Error Rate	< 1% of total requests	720ms	Passed
Security	API keys are not exposed in logs	Clean	Passed
Scheduled Task	Auto-restock runs every 10 min	3.2%	Passed

5. Test Case Specifications (Drafts)

All test cases below are designed for InvAgent's core workflow:
Unstructured user input → GLM reasoning → Tool/API orchestration → State update → Structured output

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario that triggers an error in the system, and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case - Complete Manual Restock Workflow (ProcurementController.restock() + SupplierController.getBestSupplier() + InventoryController.needsRestock())	Verify a complete restock workflow from trigger to successful order creation when stock is below threshold	1. Set product id=1 with stockLevel=5, restockThreshold=30 2. Ensure suppliers exist for product id=1 in product_supplier table (at least 2 suppliers with different ratings) 3. Send POST /procurement/restock/1	Response contains: "Restock triggered for product ID 1	Response contains: "Restock triggered for product ID 1

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			4. Check response message 5. Verify logs for each step		
TC-02	Negative Case - Remove Stock with Insufficient Quantity (InventoryController.removeStock())	Verify system rejects removal when requested quantity exceeds available stock, preventing negative inventory	1. Set product id=1 current stockLevel = 10 (via database or prior addStock call) 2. Create Product object with id=1 3. Send POST /inventory/remove?quantity=100 with Product JSON (100 > available 10) 4. Check response message 5. Verify stock level remains unchanged 6. Verify error is logged	Response starts with: "Failed to remove stock:" Error message contains insufficient stock indication Logger contains: "[ERROR REMOVE STOCK CONTROLLER]" with exception message Stock level for product id=1 remains 10 (not -90 or 0) inventoryService.removeStock() may throw exception which controller catches	Response starts with: "Failed to remove stock:" Error message contains insufficient stock indication Logger contains: "[ERROR REMOVE STOCK CONTROLLER]" with exception message Stock level for product id=1 remains 10 (not -90 or 0) inventoryService.removeStock() may throw exception which controller catches
TC-03	Performance - Auto-Restock Scheduled Task Execution Time (@Scheduled fixedRate=600000 on ProcurementController.autoRestock())	Measure the execution time of the scheduled auto-restock task to ensure it completes quickly and doesn't block subsequent scheduled executions	1. Ensure product id=1 has stockLevel below restockThreshold (e.g., stock=5, threshold=30) 2. Ensure suppliers exist for product id=1 3. Manually trigger autoRestock() method or wait	Average execution time < 500 milliseconds All operations within task complete (stock check, supplier lookup, restock trigger) Task does not exceed the 10-minute interval (no overlap)	Average execution time < 500 milliseconds All operations within task complete (stock check, supplier lookup, restock trigger) Task does not exceed the 10-minute interval (no overlap)

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			for scheduled run 4. Measure time from method entry to exit using System.currentTimeMillis() or logging timestamps 5. Run measurement 5 times and calculate average	Logger contains "[AUTO RESTOCK]" with product, supplier, quantity No timeout or hanging operations	Logger contains "[AUTO RESTOCK]" with product, supplier, quantity No timeout or hanging operations
TC-04	Concurrency - Concurrent Add Stock Operations (InventoryController.addStock())	Verify that multiple concurrent add stock requests to the same product do not cause race conditions or lost updates, ensuring data integrity	. Set initial stockLevel for product id=1 = 0 2. Prepare 10 concurrent POST requests to /inventory/add?quantity=10 with same Product JSON (id=1) 3. Execute all 10 requests simultaneously using threading tool (e.g., ExecutorService, JMeter, or Postman runner) 4. Wait for all requests to complete 5. Query final stockLevel for product id=1	All 10 requests return "Stock added successfully" Final stockLevel = 100 (10 requests × 10 quantity each) No "Failed to add stock" responses No optimistic locking exceptions in logs Logger shows 10 separate "[REQUEST ADD STOCK]" entries Race condition properly handled (requires @Transactional or synchronized method)	All 10 requests return "Stock added successfully" Final stockLevel = 100 (10 requests × 10 quantity each) No "Failed to add stock" responses No optimistic locking exceptions in logs Logger shows 10 separate "[REQUEST ADD STOCK]" entries Race condition properly handled (requires @Transactional or synchronized method)

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			6. Check all responses for errors		

6. AI Output & Boundary Testing (Drafts)

6.1. **Prompt/Response Test Pairs**

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	GET /suppliers/1/best (No body required)	Response: JSON object HTTP Status: 200 OK Contains fields: id, name, rating, leadTime Rating is highest among all suppliers for product 1 Logger contains: [REQUEST] Best supplier for product 1		

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2. Hallucinating Handling

This section describes how InvAgent detects and mitigates hallucinated responses from GLM.

Aspect	Details
Definition of Hallucination for InvAgent	GLM fabricates: ● Stock quantity not matching actual database ● Supplier names not in approved list ● PO confirmation without actual API call ● Approval status that never occurred
Detection Method #1: Schema Validation	GLM output must conform to JSON schema containing: {action: string, parameters: object, confidence: float}. If the schema is invalid → reject and retry.
Detection Method #2: Cross-Validation with APIs	Before executing any tool call, the system verifies: - Stock quantity returned by GLM matches the actual API response (if GLM pre-filled it) - Supplier name exists in list_suppliers() results - PO creation returns a valid PO number from the ERP system
Detection Method #3: Confidence Scoring	GLM returns logprobs/confidence score with each response. Threshold: ● Confidence ≥ 0.8 → Accept ● 0.5 ≤ Confidence < 0.8 → Retry with lower temperature (0.3) ● Confidence < 0.5 → Reject and escalate to human
Mitigation Strategy #1: Retry with Lower Temperature	On suspected hallucination, resend prompt with temperature reduced from 0.7 to 0.2 for more deterministic output
Mitigation Strategy #2: Constrained Prompting	System prompt includes: "Never invent stock quantities. Never invent supplier names. If unsure, ask for clarification."
Mitigation Strategy #3: Human Escalation	After 2 consecutive hallucinations → pause workflow and notify human: "GLM uncertain. Please review and provide correct information."
Mitigation Strategy #4: Audit Logging	All GLM inputs, outputs, confidence scores, and retry attempts are logged for offline analysis and prompt improvement

Important Note:

© UMHackathon 2026

Email: umhackathon@um.edu.my

These AI tests are executed iteratively throughout development, not only at the end. Each prompt engineering change, temperature adjustment, or schema update triggers the re-execution of all AI-XX test cases. The goal is to maintain the hallucination rate below 5% across all production prompts.

© UMHackathon 2026

Email: umhackathon@um.edu.my